Fritidslagret: rapporter med AI-stöd

Dokumentation av AI-användningen på grenen `feature-reports`

Projekt	`se.chasacademy.fritidslagret`, gren `feature-reports`
Datum	2026-09-30
AI-assistent	Claude Code (Anthropic), modell Claude Opus 5.5
Arbetssätt	NOEDIT under analys och verifiering (i en kopia). På begäran skrev AI:n sedan in förslaget i projektet (2026-09-30 14:25) så att det kunde inspekteras i IntelliJ.
Tester med förslaget	**test** 42 (1 överhoppat) och **ui-test** 1, alla gröna
Mutationskontroll	13 av 13 medvetna fel fångades av det test som påstår sig täcka dem
Slutversionen	**behållet** 28 · **ändrat** 24 · **förkastat** 3 · **eget** 2 enheter. Tester: **test** 40 (1 överhoppat) och **ui-test** 1, alla gröna
Genererad	2026-09-30 15:40

1. Vilken AI-assistent som användes

Claude Code från Anthropic är en AI-assistent för programmering som körs i terminalen och kan läsa filer och köra kommandon. Modellen var **Claude Opus 5.5** (`claude-opus-5-5`, 1M tokens kontext) med ansträngningsnivån **max** (`/effort max`).

Under analysen och verifieringen arbetade assistenten i **NOEDIT-läge**: den fick läsa projektet och köra git-kommandon som bara läser, men inte ändra några filer i projektet. Allt som skulle byggas och testas kördes i en kopia (`~/.cache/claude-fritidslagret-rapporter`). Därefter bad vi uttryckligen assistenten skriva in förslaget i projektet (`feature-reports`, 2026-09-30 14:25), så att det kunde inspekteras i IntelliJ. Där blev förslaget utgångsläget: allt som behålls, ändras och tas bort efter det mäts av en watcher (avsnitt 4). Testerna kördes också på en exakt kopia av projektet med förslaget inskrivet, och alla var gröna.

2. Vilken prompt som användes

Innan prompten gavs ett kontextmeddelande i samma session: `NOEDIT, fritidslagret, branch feature-reports`. AI:n svarade då med läget i repot (grenen fanns bara lokalt och hade inga egna commits). Därefter sattes `/effort max` och den här prompten klistrades in ordagrant:

```
Gör en rapport som visar dessa kriterium(Konsolutskrift):

* Aktiva lån
* Mest utlånad utrustning
* Utrustning som aldrig lånats ut
* Medlemmar med flest lån
* Tillgänglig utrustning

Använd:

* SOLID-principer
* Följ projektets allmänna struktur
* Genererara RELEVANTA tester och dubbekolla att dom faktiskt testar det dom föreslår.

Generera sen en pdf för detta och dokumentera:
* Vilken AI-assistent ni använde.
```

```
* Vilken prompt ni använde.
* Vad AI:n föreslog.

- Här behövs en watcher för att kunna se vad vi behåller samt ändrar
    * Vad ni valde att behålla.
    * Vad ni valde att ändra.
    * Vad ni valde att förkasta.

- Här behövs en till watcher som frågar användaren om kompletterande uppgifter när ovan är klart
    * Varför ni gjorde dessa val.
```

3. Vad AI:n föreslog

Förslaget lägger till valet **6. Rapporter** i huvudmenyn. Det skriver ut en rapport i konsolen med de fem kriterierna. Beräkningarna ligger i en ny `ReportService`. Utskriften ligger i `ReportMenu`, som går igenom fem små sektionsklasser bakom ett gemensamt interface. Förslaget har 12 nya tester: 8 för beräkningarna, 2 för `LoanService` och 2 för själva konsolutskriften. All kod finns i bilaga A.

3.1 Filer och ansvar

Fil	Ny/ändrad	Ansvar
`service/ReportService`	ny	Räknar fram rapportens data. Skriver aldrig ut något.
`ui/ReportMenuSection`	ny	Interface: en sektion har en rubrik och en lista rader.
`ui/ReportMenuActiveLoans` `ui/ReportMenuMostBorrowedEquipment` `ui/ReportMenuNeverBorrowedEquipment` `ui/ReportMenuMembersWithMostLoans` `ui/ReportMenuAvailableEquipment`	nya	En klass per kriterium. Formaterar raderna för sitt kriterium.
`ui/ReportMenu`	ny	Skriver ut alla sektioner i tur och ordning, med "Inget att visa" för tomma.
`service/LoanService`	ändrad	+ `getAllLoans()`: alla lån, även återlämnade.
`ui/MainMenu`	ändrad	Nytt val `6. Rapporter`, och `Avsluta` flyttas till 7.
`Main`	ändrad	Skapar `ReportService` och skickar in den i `MainMenu`.
`test …/ReportServiceTest`	ny	8 tester av beräkningarna.
`test …/ReportMenuTest`	ny	2 tester av konsolutskriften.
`test …/LoanServiceTest`	ändrad	+ 2 tester (getAllLoans, aktiva lån).
`test …/EquipmentMenuDeleteTest`	ändrad	Avslutar med `7` i stället för `6`.

Förslaget följer projektets struktur: paketen `service` och `ui`, konstruktorinjektion, prefixet `ReportMenu…` för delarna av en meny (som `EquipmentMenuAdd` och `LoanMenuList`), `IO.println` och `String.format`, svenska texter i gränssnittet och engelska namn i koden. Testerna är skrivna med Arrange/Act/Assert, där alla `when(...)` ligger i Arrange.

3.2 Så tolkades kriterierna

Kriterium	Tolkning i förslaget	Motivering
Aktiva lån	Lån där `isLoanActive()` är sant, sorterade på låne-id. Befintliga `getAllActiveLoansSortedById()` återanvänds.	Metoden finns redan och används av lånemenyn, så båda visar samma sak.
Mest utlånad utrustning	Flest lån totalt, även återlämnade. Vid lika antal visas alla. Utrustning utan lån visas aldrig.	"Mest utlånad" gäller historiken, inte vad som är utlånat just nu.
Utrustning som aldrig lånats ut	Utrustning utan ett enda lån i historiken, oavsett om den är tillgänglig eller inte.	`8-people-tent` i Startup är otillgängligt men aldrig utlånat. En cykel som lämnats tillbaka är tillgänglig men har varit utlånad.

Kriterium	Tolkning i förslaget	Motivering
Medlemmar med flest lån	Alla medlemmar med minst ett lån, flest först. Återlämnade lån räknas. Vid lika antal sorteras de på medlems-id.	`Member.getActiveLoans()` räknar bara pågående lån (högst 3). Sorteringen på id gör utskriften stabil, eftersom `Map.copyOf` i repona inte har någon fast ordning.
Tillgänglig utrustning	`isAvailable()` är sant, sorterat på id, med max lånetid.	Samma uppgift som statistikmenyn visar, fast som en lista.

3.3 SOLID

	Så syns principen i förslaget
S	`ReportService` räknar, sektionerna formaterar och `ReportMenu` skriver ut. `StatisticsService` gör i dag både beräkning och `IO.println`. Det mönstret upprepas inte.
O	Ett nytt kriterium blir en ny klass som implementerar `ReportMenuSection` plus en rad i listan i `ReportMenu`. `showReport()` ändras inte.
L	Alla fem sektionerna skrivs ut av samma loop och kan byta plats utan specialfall.
I	`ReportMenuSection` har bara `getTitle()` och `getRows()`, alltså det enda menyn behöver.
D	Utskriftsloopen i `ReportMenu` beror på interfacet och inte på de konkreta sektionerna. Tjänsterna skickas in via konstruktorn som i resten av projektet. *Avvägning:* projektets tjänster saknar interface, så ReportService tar `LoanService` och `EquipmentService` direkt. Att ge alla tjänster interface vore en större omstrukturering än uppgiften.

3.4 Tester och vad de faktiskt bevisar

Ett test är bara relevant om det blir rött när det beteende det beskriver går sönder. Därför bröts koden medvetet på 13 sätt, ett i taget (tabell 3.5). Varje test nedan blev rött av minst en mutation som bryter just det som testnamnet påstår.

Test	Vad testet visar	Rött vid mutation
`ReportServiceTest shouldCountReturnedLoansWhenCountingLoansForEquipment`	Återlämnade lån räknas när lånen per utrustning räknas.	M1
`ReportServiceTest shouldReturnEquipmentWithMostLoans`	Den utrustning som har flest lån väljs, även när den inte ligger först i id-ordning.	M1, M4
`ReportServiceTest shouldReturnAllEquipmentSharingTheHighestLoanCount`	Vid lika antal lån visas alla som delar förstaplatsen, men inte utrustning utan lån.	M1, M3
`ReportServiceTest shouldNotReturnAnyMostBorrowedEquipmentWhenNothingHasBeenLoaned`	Utan några lån finns ingen "mest utlånad", och listan blir tom.	M2
`ReportServiceTest shouldReturnNeverBorrowedEquipmentEvenWhenItIsUnavailable`	"Aldrig utlånad" bygger på historiken: ett otillgängligt tält utan lån är med, men en återlämnad cykel är inte med.	M1, M5
`ReportServiceTest shouldSortMembersWithMostLoansFirstCountingReturnedLoans`	Medlemmar sorteras med flest lån först, och återlämnade lån räknas.	M6, M7

Test	Vad testet visar	Rött vid mutation
`ReportServiceTest shouldSortMembersWithSameLoanCountByMemberId`	Medlemmar med lika många lån sorteras på medlems-id, så att utskriften blir stabil.	M8
`ReportServiceTest shouldOnlyReturnAvailableEquipment`	Bara tillgänglig utrustning är med, i id-ordning.	M9
`LoanServiceTest shouldIncludeReturnedLoansWhenGettingAllLoans`	getAllLoans() tar med återlämnade lån, och det är dem historiken bygger på.	M10
`LoanServiceTest shouldNotListReturnedLoansAsActiveLoans`	Källan till "Aktiva lån" hoppar över återlämnade lån.	M11
`ReportMenuTest shouldPrintEachReportSectionTitleFollowedByItsRows`	Konsolutskriften: varje rubrik följs direkt av sina egna rader.	M1, M5, M6, M9, M12
`ReportMenuTest shouldPrintNothingToShowUnderSectionsWithoutRows`	En tom sektion skriver "Inget att visa" i stället för en tom rubrik.	M12, M13

Ett test av `getActiveLoans()` med en mockad `LoanService` skulle bara ha testat mocken. Därför testas källan (`getAllActiveLoansSortedById()`) i stället, mot riktig `LoanService`-logik. Att listan är sorterad garanteras av returtypen `SortedMap`, så inget test påstår sig kontrollera det.

3.5 Verifiering

- **Före förslaget:** `test` 30 tester (1 överhoppat) och `ui-test` 1, alla gröna.
- **Med förslaget:** `test` 42 (1 överhoppat) och `ui-test` 1, alla gröna. Samma resultat ovanpå `main` efter PR #5 (`f313ca1`), eftersom PR #5 inte rör någon av förslagets filer.
- **Rött läge:** de nya testerna mot originalkoden ger 70 kompileringsfel (`ReportService`, `ReportMenu` och `getAllLoans` finns inte än).
- **Mutationer:** 13 av 13 fångade:

#	Medvetet fel i koden	Fil	Röda tester (förväntat test i fetstil)
M1	Räknar bara aktiva lån per utrustning	`ReportService.java`	shouldReturnNeverBorrowedEquipmentEvenWhenItIsUnavailable shouldReturnAllEquipmentSharingTheHighestLoanCount shouldReturnEquipmentWithMostLoans **shouldCountReturnedLoansWhenCountingLoansForEquipment** shouldPrintEachReportSectionTitleFollowedByItsRows
M2	Startvärdet för högsta antal är 0 i stället för 1	`ReportService.java`	**shouldNotReturnAnyMostBorrowedEquipmentWhenNothingHasBeenLoaned**
M3	Bara en utrustning visas när flera har lika många lån	`ReportService.java`	**shouldReturnAllEquipmentSharingTheHighestLoanCount**
M4	Listan töms inte när ett nytt högsta antal hittas	`ReportService.java`	**shouldReturnEquipmentWithMostLoans**
M5	"Aldrig utlånad" räknas som "tillgänglig just nu"	`ReportService.java`	**shouldReturnNeverBorrowedEquipmentEvenWhenItIsUnavailable** shouldPrintEachReportSectionTitleFollowedByItsRows

#	Medvetet fel i koden	Fil	Röda tester (förväntat test i fetstil)
M6	Medlemmarnas lån räknas bara på aktiva lån	`ReportService.java`	**shouldSortMembersWithMostLoansFirstCountingReturnedLoans** shouldPrintEachReportSectionTitleFollowedByItsRows
M7	Medlemmar sorteras stigande (minst lån först)	`ReportService.java`	**shouldSortMembersWithMostLoansFirstCountingReturnedLoans**
M8	Ingen sortering på medlems-id vid lika antal lån	`ReportService.java`	**shouldSortMembersWithSameLoanCountByMemberId**
M9	Tillgänglig utrustning filtreras inte	`ReportService.java`	**shouldOnlyReturnAvailableEquipment** shouldPrintEachReportSectionTitleFollowedByItsRows
M10	getAllLoans() tar bara med aktiva lån	`LoanService.java`	**shouldIncludeReturnedLoansWhenGettingAllLoans**
M11	"Aktiva lån" tar med återlämnade lån	`LoanService.java`	**shouldNotListReturnedLoansAsActiveLoans**
M12	En sektion (Medlemmar med flest lån) saknas i rapporten	`ReportMenu.java`	**shouldPrintEachReportSectionTitleFollowedByItsRows** **shouldPrintNothingToShowUnderSectionsWithoutRows**
M13	Tom sektion skriver ingenting under rubriken	`ReportMenu.java`	**shouldPrintNothingToShowUnderSectionsWithoutRows**

3.6 Konsolutskrift från appen

Körd med Startup-datan, där valet 6 och sedan 7 matas in:

```
Rapport
-----------------
Aktiva lån:
    Lån-ID: 1000, Medlem: Ylva, Artikel: SkiSet-for-Children, Lånedatum: 2026-09-30, Senaste datum för
återlämning: 2026-10-03
    Lån-ID: 1001, Medlem: Ylva, Artikel: SkiSet-for-Small-Children, Lånedatum: 2026-09-30, Senaste datum för
återlämning: 2026-10-10

Mest utlånad utrustning:
    Id: 3 - SkiSet-for-Children (SkiSet), 1 lån
    Id: 4 - SkiSet-for-Small-Children (SkiSet), 1 lån

Utrustning som aldrig lånats ut:
    Id: 1 - Bicycle (Bike)
    Id: 2 - 8-people-tent (Tent)
    Id: 5 - Motorcycle (Bike)

Medlemmar med flest lån:
    Medlems-Id: 1000 - Ylva, 2 lån

Tillgänglig utrustning:
    Id: 1 - Bicycle (Bike), max 30 dagar
    Id: 5 - Motorcycle (Bike), max 180 dagar
```

Samma app efter att lån 1000 lämnats tillbaka och samma skidset lånats igen. Historiken räknas (2 lån på skidsetet, 3 lån för Ylva), men det återlämnade lånet är inte med bland de aktiva:

```
Rapport
-----------------
Aktiva lån:
    Lån-ID: 1001, Medlem: Ylva, Artikel: SkiSet-for-Small-Children, Lånedatum: 2026-09-30, Senaste datum för
återlämning: 2026-10-10
    Lån-ID: 1002, Medlem: Ylva, Artikel: SkiSet-for-Children, Lånedatum: 2026-09-30, Senaste datum för
återlämning: 2026-10-03

Mest utlånad utrustning:
```

```
    Id: 3 - SkiSet-for-Children (SkiSet), 2 lån

Utrustning som aldrig lånats ut:
    Id: 1 - Bicycle (Bike)
    Id: 2 - 8-people-tent (Tent)
    Id: 5 - Motorcycle (Bike)

Medlemmar med flest lån:
    Medlems-Id: 1000 - Ylva, 3 lån

Tillgänglig utrustning:
    Id: 1 - Bicycle (Bike), max 30 dagar
    Id: 5 - Motorcycle (Bike), max 180 dagar
```

3.7 Upptäckt på vägen (inte ändrat i förslaget)

1. **Statistikmenyns "Antal aktiva lån" räknar alla lån som någonsin skapats.** `LoanService.getActiveLoanCount()` returnerar storleken på hela lånedatabasen. Efter en återlämning visar menyn fortfarande 2, fast bara 1 lån är aktivt (återskapat i appen). Rapportens "Aktiva lån" räknar rätt. Koden kommer från statistikgrenen och ändras därför inte här.
2. **`LoanService.getMostBorrowedEquipment()` väljer en slumpmässig utrustning när flera har lika många lån** (`HashMap` med `Equipment` som nyckel utan `hashCode`, så ordningen följer objektidentiteten).
3. **Menyerna snurrar för alltid när indata tar slut.** `IO.readln()` returnerar `null`, `Integer.parseInt(null)` kastar `NumberFormatException` och den fångas. I ett UI-test växer den fångade utskriften tills testprocessen dör av *Java heap space* (efter cirka 9 s, återskapat). Därför måste `EquipmentMenuDeleteTest` avsluta med 7 när Rapporter tar plats 6. Annars fallerar `ui-test`, och med den `run` och `jar`.
4. **De statiska id-räknarna gör tester ordningsberoende i samma JVM.** Om rapportens konsoltest hade körts som UI-test före `EquipmentMenuDeleteTest` hade Startup inte hittat medlem 1000, och `Main.main` hade kraschat med `NullPointerException` (återskapat). Därför heter testet `ReportMenuTest` och körs i den vanliga `test`-tasken.

4. Vad som behölls, ändrades och förkastades

Förslaget skrevs in i projektet av AI:n kl. 14:25 och var utgångsläget (behållet 55). **Watcher 1** följde sedan projektet medan förslaget granskades och bearbetades i IntelliJ. Varje metod, konstruktor och klass med fält jämfördes med förslaget. Blanksteg, radbrytningar och CRLF räknas inte, och omdöpta eller flyttade delar känns igen på likheten i koden. Slutläget frystes 2026-09-30 15:39:03 på grenen `feature-reports`, oincheckat ovanpå `f313ca1`: **behållet 28** · **ändrat 24** · **förkastat 3** · **eget 2**

4.1 Översikt

Klass i förslaget	Enheter
`Main`	behållet 1
`LoanService`	behållet 1
`ReportService`	behållet 8 · ändrat 1
`MainMenu`	behållet 3
`ReportMenu`	behållet 2 · ändrat 1
`ReportMenuActiveLoans` → `ReportSectionActiveLoans`	ändrat 4
`ReportMenuAvailableEquipment` → `ReportSectionAvailableEquipment`	ändrat 4
`ReportMenuMembersWithMostLoans` → `ReportSectionMembersWithTopThreeMostTotalLoans`	ändrat 4
`ReportMenuMostBorrowedEquipment` → `ReportSectionMostBorrowedEquipment`	ändrat 4
`ReportMenuNeverBorrowedEquipment` → `ReportSectionNeverBorrowedEquipment`	ändrat 4
`ReportMenuSection`	behållet 3
`LoanServiceTest`	behållet 2
`ReportServiceTest`	behållet 7 · ändrat 2
`EquipmentMenuDeleteTest`	behållet 1
`ReportMenuTest`	förkastat 3

4.2 Ändrat

Sektionsklasserna döptes om från ReportMenu… till ReportSection…

- `ReportMenu (konstruktor)`
- `ReportMenuActiveLoans (klass och fält)` *(flyttad/omdöpt till ReportSectionActiveLoans (klass och fält) (likhet 91%))*
- `ReportMenuActiveLoans (konstruktor)` *(flyttad/omdöpt till ReportSectionActiveLoans (konstruktor) (likhet 93%))*
- `ReportMenuActiveLoans.getTitle()` *(flyttad/omdöpt till ReportSectionActiveLoans.getTitle())*
- `ReportMenuAvailableEquipment (klass och fält)` *(flyttad/omdöpt till ReportSectionAvailableEquipment (klass och fält) (likhet 91%))*
- `ReportMenuAvailableEquipment (konstruktor)` *(flyttad/omdöpt till ReportSectionAvailableEquipment (konstruktor) (likhet 93%))*

- `ReportMenuAvailableEquipment.getTitle()` *(flyttad/omdöpt till ReportSectionAvailableEquipment.getTitle())*
- `ReportMenuMembersWithMostLoans (klass och fält)` *(flyttad/omdöpt till ReportSectionMembersWithTopThreeMostTotalLoans (klass och fält) (likhet 91%))*
- `ReportMenuMembersWithMostLoans (konstruktor)` *(flyttad/omdöpt till ReportSectionMembersWithTopThreeMostTotalLoans (konstruktor) (likhet 93%))*
- `ReportMenuMostBorrowedEquipment (klass och fält)` *(flyttad/omdöpt till ReportSectionMostBorrowedEquipment (klass och fält) (likhet 91%))*
- `ReportMenuMostBorrowedEquipment (konstruktor)` *(flyttad/omdöpt till ReportSectionMostBorrowedEquipment (konstruktor) (likhet 93%))*
- `ReportMenuMostBorrowedEquipment.getTitle()` *(flyttad/omdöpt till ReportSectionMostBorrowedEquipment.getTitle())*
- `ReportMenuMostBorrowedEquipment.getRows()` *(flyttad/omdöpt till ReportSectionMostBorrowedEquipment.getRows())*
- `ReportMenuNeverBorrowedEquipment (klass och fält)` *(flyttad/omdöpt till ReportSectionNeverBorrowedEquipment (klass och fält) (likhet 91%))*
- `ReportMenuNeverBorrowedEquipment (konstruktor)` *(flyttad/omdöpt till ReportSectionNeverBorrowedEquipment (konstruktor) (likhet 93%))*
- `ReportMenuNeverBorrowedEquipment.getTitle()` *(flyttad/omdöpt till ReportSectionNeverBorrowedEquipment.getTitle())*
- `ReportMenuNeverBorrowedEquipment.getRows()` *(flyttad/omdöpt till ReportSectionNeverBorrowedEquipment.getRows())*

`ReportMenu (konstruktor)`

```
--- förslaget
+++ ert projekt
@@ -2,9 +2,9 @@
     // A new criterion becomes a new class in this list, showReport() stays the same
     this.reportSections = List.of(
-            new ReportMenuActiveLoans(reportService),
-            new ReportMenuMostBorrowedEquipment(reportService),
-            new ReportMenuNeverBorrowedEquipment(reportService),
-            new ReportMenuMembersWithMostLoans(reportService),
-            new ReportMenuAvailableEquipment(reportService)
+            new ReportSectionActiveLoans(reportService),
+            new ReportSectionMostBorrowedEquipment(reportService),
+            new ReportSectionNeverBorrowedEquipment(reportService),
+            new ReportSectionMembersWithTopThreeMostTotalLoans(reportService),
+            new ReportSectionAvailableEquipment(reportService)
     );
 }
```

`ReportMenuActiveLoans (klass och fält)`

```
--- förslaget
+++ ert projekt
@@ -1,2 +1,2 @@
-public class ReportMenuActiveLoans implements ReportMenuSection {
+public class ReportSectionActiveLoans implements ReportMenuSection {
 private final ReportService reportService;
```

> **Motivering (fråga 1 i avsnitt 5):** "Vi döpte om dom pga av att dom inte var menyer utan var en sektion i rapporten"

"Medlemmar med flest lån" blev en topp 3-lista

- `ReportService.getMembersWithMostLoans()` *(flyttad/omdöpt till ReportService.getTopThreeMembersWithMostTotalLoans() (likhet 75%))*
- `ReportMenuMembersWithMostLoans.getTitle()` *(flyttad/omdöpt till ReportSectionMembersWithTopThreeMostTotalLoans.getTitle() (likhet 92%))*
- `ReportMenuMembersWithMostLoans.getRows()` *(flyttad/omdöpt till ReportSectionMembersWithTopThreeMostTotalLoans.getRows() (likhet 99%))*
- `ReportServiceTest.shouldSortMembersWithMostLoansFirstCountingReturnedLoans()`
- `ReportServiceTest.shouldSortMembersWithSameLoanCountByMemberId()`

ReportService.getMembersWithMostLoans()

```
--- förslaget
+++ ert projekt
@@ -1,3 +1,3 @@
-public List<Member> getMembersWithMostLoans() {
+public List<Member> getTopThreeMembersWithMostTotalLoans() {
     List<Member> membersWithLoans = new ArrayList<>();
     for (Loan loan : loanService.getAllLoans()) {
@@ -8,4 +8,14 @@
     membersWithLoans.sort(Comparator.comparing(this::getLoanCountForMember).reversed()
             .thenComparing(Member::getMemberId));
-    return membersWithLoans;
+
+    List<Member> topThreeMembersWithMostTotalLoans = new ArrayList<Member>();
+    for (int i = 0; i <= 2; i++) {
+        if (membersWithLoans.size() <= i) {
+            continue;
+        }
+
+        topThreeMembersWithMostTotalLoans.add(membersWithLoans.get(i));
+    }
+
+    return topThreeMembersWithMostTotalLoans;
 }
```

ReportMenuMembersWithMostLoans.getTitle()

```
--- förslaget
+++ ert projekt
@@ -1,4 +1,4 @@
 @Override
 public String getTitle() {
-    return "Medlemmar med flest lån";
+    return "Medlem/-mar med flest totalt antal lån(Topp 3)";
 }
```

ReportMenuMembersWithMostLoans.getRows()

```
--- förslaget
+++ ert projekt
@@ -2,5 +2,5 @@
 public List<String> getRows() {
     List<String> rows = new ArrayList<>();
-    for (Member member : reportService.getMembersWithMostLoans()) {
+    for (Member member : reportService.getTopThreeMembersWithMostTotalLoans()) {
         rows.add(String.format("Medlems-Id: %d - %s, %d lån",
                 member.getMemberId(), member.getName(), reportService.getLoanCountForMember(member)));
```

ReportServiceTest.shouldSortMembersWithMostLoansFirstCountingReturnedLoans()

```
--- förslaget
+++ ert projekt
@@ -19,5 +19,5 @@

     // Act
-    List<Member> membersWithMostLoans = reportService.getMembersWithMostLoans();
+    List<Member> membersWithMostLoans = reportService.getTopThreeMembersWithMostTotalLoans();

     // Assert
```

ReportServiceTest.shouldSortMembersWithSameLoanCountByMemberId()

```
--- förslaget
+++ ert projekt
@@ -15,5 +15,5 @@

     // Act
-    List<Member> membersWithMostLoans = reportService.getMembersWithMostLoans();
+    List<Member> membersWithMostLoans = reportService.getTopThreeMembersWithMostTotalLoans();
```

```
    // Assert
```

> **Motivering (fråga 2 i avsnitt 5):** "Vi tyckte det inte behövdes mer än topp3 lista eftersom annars alla medlemmar skulle komma med och rangordnas"

"Försenad: Ja/Nej" lades till på aktiva lån

- `ReportMenuActiveLoans.getRows()` *(flyttad/omdöpt till ReportSectionActiveLoans.getRows() (likhet 92%))*

`ReportMenuActiveLoans.getRows()`

```
--- förslaget
+++ ert projekt
@@ -3,5 +3,5 @@
     List<String> rows = new ArrayList<>();
     for (Loan loan : reportService.getActiveLoans()) {
-        rows.add(loan.toString());
+        rows.add(loan.toString() + ", Försenad: " + reportService.isDelayed(loan));
     }
     return rows;
```

> **Motivering (fråga 3 i avsnitt 5):** "Tyckte det va relevant information i en rapport där man listar aktiva lån"

"max" blev "max-lånetid" i Tillgänglig utrustning

- `ReportMenuAvailableEquipment.getRows()` *(flyttad/omdöpt till ReportSectionAvailableEquipment.getRows() (likhet 99%))*

`ReportMenuAvailableEquipment.getRows()`

```
--- förslaget
+++ ert projekt
@@ -3,5 +3,5 @@
     List<String> rows = new ArrayList<>();
     for (Equipment equipment : reportService.getAvailableEquipment()) {
-        rows.add(String.format("Id: %d - %s (%s), max %d dagar",
+        rows.add(String.format("Id: %d - %s (%s), max-lånetid %d dagar",
                 equipment.getEquipmentId(), equipment.getEquipmentName(), equipment.getType(),
                 equipment.getMaxLoanDays()));
```

> **Motivering (fråga 4 i avsnitt 5):** "Otydlighet i endast max "30" dagar, 30 dagar vadå max?"

4.3 Förkastat

- `ReportMenuTest (klass och fält)`
- `ReportMenuTest.shouldPrintEachReportSectionTitleFollowedByItsRows()`
- `ReportMenuTest.shouldPrintNothingToShowUnderSectionsWithoutRows()`

> **Motivering (fråga 6 i avsnitt 5):** "Överkurs för denna inlämningsuppgift"

> **Följd:** inget test kontrollerar längre själva konsolutskriften. I mutationskontrollen av förslaget var det bara `ReportMenuTest` som fångade en saknad sektion (M12) och en tom sektion utan "Inget att visa" (M13). I slutversionen passerar samma fel alla tester (F2 och F3 i 4.7).

4.4 Eget, inte från förslaget

Startup.startUp() (ändrad, `Startup.java`)

```
--- före
+++ nu
@@ -4,8 +4,13 @@
     // Equipment start
```

```
      equipmentService.addEquipment("Bicycle", true, 30, BIKE);
-     equipmentService.addEquipment("8-people-tent", false, 90, TENT);
+     equipmentService.addEquipment("8-people-tent", true, 90, TENT);
      equipmentService.addEquipment("SkiSet-for-Children", true, 3, SKISET);
      equipmentService.addEquipment("SkiSet-for-Small-Children", true, 10, SKISET);
      equipmentService.addEquipment("Motorcycle", true, 180, BIKE);
+     equipmentService.addEquipment("Motorcycle2", true, 20, BIKE);
+     equipmentService.addEquipment("Motorcycle3", true, 20, BIKE);
+     equipmentService.addEquipment("Motorcycle4", true, 20, BIKE);
+     equipmentService.addEquipment("Motorcycle5", true, 20, BIKE);
+
      // Users start

@@ -13,10 +18,18 @@
      memberService.addMember("Nisse", REVOKED);
      memberService.addMember("Bosse", INACTIVE);
-     memberService.addMember("NoLoanInitMember", ACTIVE);
+     memberService.addMember("NoLoanMember", ACTIVE);
+     memberService.addMember("NoLoanMember2", ACTIVE);
+     memberService.addMember("NoLoanMember3", ACTIVE);
+     memberService.addMember("NoLoanMember4", ACTIVE);
+

      // Loans start
-     //loanService.startLoan(2, memberService.getMember(1000));
-     loanService.startLoan(3, memberService.getMember(1000));
-     loanService.startLoan(4, memberService.getMember(1000));
-     //LoanService.startLoan(1, MemberService.getMember(1000));
+     loanService.startLoan(2, memberService.getMember(1000));
+     loanService.startLoan(3, memberService.getMember(1003));
+     loanService.startLoan(4, memberService.getMember(1004));
+     loanService.startLoan(1, memberService.getMember(1005));
+
+     loanService.returnLoan(loanService.getLoanById(1000));
+     loanService.startLoan(2, memberService.getMember(1000));
+
  }
```

> **Motivering (fråga 5 i avsnitt 5):** "Behövde mer data för att ge konkret bild hur rapporten visades vid olika mängder data"

ReportService.isDelayed() (ny, `ReportService.java`)

```
public String isDelayed(Loan loan) {
    boolean delayed = loanService.isDelayed(loan);
    if (delayed) {
        return "Ja";
    }
    return "Nej";
}
```

> **Motivering (fråga 3 i avsnitt 5):** "Tyckte det va relevant information i en rapport där man listar aktiva lån"

4.5 Behållet

Klass	Behållna enheter
`Main`	`main()`
`LoanService`	`getAllLoans()`
`ReportService`	`(klass och fält)`, `(konstruktor)`, `getActiveLoans()`, `getMostBorrowedEquipment()`, `getNeverBorrowedEquipment()`, `getAvailableEquipment()`, `getLoanCountForEquipment()`, `getLoanCountForMember()`
`MainMenu`	`(klass och fält)`, `(konstruktor)`, `mainMenu()`

Klass	Behållna enheter
`ReportMenu`	`(klass och fält)`, `showReport()`
`ReportMenuSection`	`(klass och fält)`, `getTitle()`, `getRows()`
`LoanServiceTest`	`shouldIncludeReturnedLoansWhenGettingAllLoans()`, `shouldNotListReturnedLoansAsActiveLoans()`
`ReportServiceTest`	`(klass och fält)`, `shouldCountReturnedLoansWhenCountingLoansForEquipment()`, `shouldReturnEquipmentWithMostLoans()`, `shouldReturnAllEquipmentSharingTheHighestLoanCount()`, `shouldNotReturnAnyMostBorrowedEquipmentWhenNothingHasBeenLoaned()`, `shouldReturnNeverBorrowedEquipmentEvenWhenItIsUnavailable()`, `shouldOnlyReturnAvailableEquipment()`
`EquipmentMenuDeleteTest`	`UIshouldThrowConfirmationFailedExceptionWhenWrongConfirmationNameIsInput()`

> **Motivering (fråga 7 i avsnitt 5):** "Såg bra ut i sin helhet vad vi kunde se och därför vart allt kvar"

4.6 AI:ns kommentarer under arbetet och vad som hände med dem

Tid	AI:ns kommentar	Vad som hände
14:48	Efter namnbytet av metoden kompilerade testerna inte: `ReportServiceTest` anropade fortfarande `getMembersWithMostLoans()` (rad 164 och 186). `ReportMenuTest` skulle också bli röd av de nya texterna.	Anropen rättades. `ReportMenuTest` togs bort i stället för att uppdateras.
14:52	`isDelayed` i `ReportService` returnerar texten "Ja"/"Nej", så gränssnittstext hamnar i servicelagret. Förslag: returnera `boolean` och gör Ja/Nej i sektionen (S i SOLID), eller använd befintliga `getStatusOfLoan`.	Inte antaget: `isDelayed` returnerar fortfarande `String`.
15:08	Topp 3-logiken är korrekt även när färre än tre medlemmar har lån. `continue` kan bli `break`. Ett test föreslogs för att bara tre medlemmar returneras.	Inte antaget: `continue` ligger kvar, och det finns inget test för topp 3-gränsen (se F1 nedan).
15:27	Sex rader i testerna för att få allt grönt: två i `ReportServiceTest`, fyra i `ReportMenuTest`.	De två i `ReportServiceTest` infördes. `ReportMenuTest` togs bort ("Överkurs för denna inlämningsuppgift").

4.7 Slutversionen

Tester (körda på en exakt kopia av arbetskopian): `test` 40 (1 överhoppat) och `ui-test` 1, alla gröna. Det är två färre än med förslaget, eftersom `ReportMenuTest` togs bort.

Mutationskontroll av slutversionen: tre medvetna fel i det som ändrades eller förlorade sitt test, plus ett kontrollfel som ska fångas.

#	Medvetet fel i slutversionen	Fil	Resultat
F1	Topp 3 blir topp 4	`ReportService.java`	**inte fångat**, alla tester gröna
F2	Sektionen med topp 3-medlemmar saknas i rapporten	`ReportMenu.java`	**inte fångat**, alla tester gröna
F3	Tom sektion skriver ingenting under rubriken	`ReportMenu.java`	**inte fångat**, alla tester gröna

#	Medvetet fel i slutversionen	Fil	Resultat
K1	Kontroll: medlemmarnas lån räknas bara på aktiva lån	`ReportService.java`	**fångat av** `shouldSortMembersWithMostLoansFirstCountingReturnedLoans`

Konsolutskrift med den nya Startup-datan, där valet 6 och sedan 7 matas in:

```
Rapport
-----------------
Aktiva lån:
    Lån-ID: 1001, Medlem: NoLoanMember, Artikel: SkiSet-for-Children, Lånedatum: 2026-09-30, Senaste datum för
återlämning: 2026-10-03, Försenad: Nej
    Lån-ID: 1002, Medlem: NoLoanMember2, Artikel: SkiSet-for-Small-Children, Lånedatum: 2026-09-30, Senaste
datum för återlämning: 2026-10-10, Försenad: Nej
    Lån-ID: 1003, Medlem: NoLoanMember3, Artikel: Bicycle, Lånedatum: 2026-09-30, Senaste datum för
återlämning: 2026-10-30, Försenad: Nej
    Lån-ID: 1004, Medlem: Ylva, Artikel: 8-people-tent, Lånedatum: 2026-09-30, Senaste datum för återlämning:
2026-12-29, Försenad: Nej

Mest utlånad utrustning:
    Id: 2 - 8-people-tent (Tent), 2 lån

Utrustning som aldrig lånats ut:
    Id: 5 - Motorcycle (Bike)
    Id: 6 - Motorcycle2 (Bike)
    Id: 7 - Motorcycle3 (Bike)
    Id: 8 - Motorcycle4 (Bike)
    Id: 9 - Motorcycle5 (Bike)

Medlem/-mar med flest totalt antal lån(Topp 3):
    Medlems-Id: 1000 - Ylva, 2 lån
    Medlems-Id: 1003 - NoLoanMember, 1 lån
    Medlems-Id: 1004 - NoLoanMember2, 1 lån

Tillgänglig utrustning:
    Id: 5 - Motorcycle (Bike), max-lånetid 180 dagar
    Id: 6 - Motorcycle2 (Bike), max-lånetid 20 dagar
    Id: 7 - Motorcycle3 (Bike), max-lånetid 20 dagar
    Id: 8 - Motorcycle4 (Bike), max-lånetid 20 dagar
    Id: 9 - Motorcycle5 (Bike), max-lånetid 20 dagar
```

4.8 Tidslinje

Watcher 1 registrerade 12 ändringstillfällen mellan händelserna nedan. Alla finns i `watch/history.jsonl`.

Tid	Händelse	Detaljer
14:25	Förslaget inskrivet av AI:n	Förslaget skrevs in i projektet av AI:n på begäran, för inspektion i IntelliJ (15 filer, CRLF)
14:55	commit `f313ca1`	Merge pull request #5 from fredrik-elias-fjax26/feature-memberMenu *(main snabbspolades in i feature-reports)* tester: test: 29 gröna, 0 röda, 1 överhoppade; ui-test: 1 gröna, 0 röda
15:27	Fas 1 klar	Gruppen meddelade att de var nöjda. Watcher 2 startade frågorna.
15:39	Slutläge fryst	behållet 28 · ändrat 24 · förkastat 3 · eget 2

5. Varför dessa val gjordes

Watcher 2 väntar tills watcher 1 är klar: grenen är pushad, arbetskopian har inga osparade ändringar och inget har hänt på 15 minuter (eller så har ni sagt att ni är klara). Då ställs frågor om varför varje ändring och varje förkastad del valdes, och även om det som behölls. Svaren står nedan ordagrant.

Fas 1 avslutades 15:27 när gruppen skrev: *"Nu har vi ändrat och lagt till och känner oss nöjd, nästa steg"*. Testerna gjordes sedan gröna igen. Därefter ställde watcher 2 en fråga per val, och gruppen svarade med egna ord.

> **1. Namnbyte: varför döpte ni om de fem sektionsklasserna från ReportMenu... till ReportSection...? Interfacet ReportMenuSection fick behålla sitt namn.**
>
> Gäller: `ReportSectionActiveLoans`, `ReportSectionMostBorrowedEquipment`, `ReportSectionNeverBorrowedEquipment`, `ReportSectionMembersWithTopThreeMostTotalLoans`, `ReportSectionAvailableEquipment`, `ReportMenu (konstruktor)`
>
> "Vi döpte om dom pga av att dom inte var menyer utan var en sektion i rapporten"

> **2. Topp 3: varför blev "Medlemmar med flest lån" en topp 3-lista med rubriken "Medlem/-mar med flest totalt antal lån(Topp 3)"? Vid lika antal på plats 3 avgör medlems-id vem som kommer med.**
>
> Gäller: `ReportService.getTopThreeMembersWithMostTotalLoans()`, `ReportSectionMembersWithTopThreeMostTotalLoans.getTitle()`, `ReportSectionMembersWithTopThreeMostTotalLoans.getRows()`
>
> "Vi tyckte det inte behövdes mer än topp3 lista eftersom annars alla medlemmar skulle komma med och rangordnas"

> **3. Försenad: varför lade ni till ", Försenad: Ja/Nej" på aktiva lån, via en egen isDelayed i ReportService som returnerar text?**
>
> Gäller: `ReportService.isDelayed()`, `ReportSectionActiveLoans.getRows()`
>
> "Tyckte det va relevant information i en rapport där man listar aktiva lån"

> **4. Lånetid: varför ändrade ni "max" till "max-lånetid" i Tillgänglig utrustning?**
>
> Gäller: `ReportSectionAvailableEquipment.getRows()`
>
> "Otydlighet i endast max "30" dagar, 30 dagar vadå max?"

> **5. Startup: varför ändrade ni demodatan (ledigt tält, fler motorcyklar och medlemmar, fler lån och en återlämning)?**
>
> Gäller: `Startup.startUp()`
>
> "Behövde mer data för att ge konkret bild hur rapporten visades vid olika mängder data"

> **6. Förkastat: varför tog ni bort ReportMenuTest?**
>
> Gäller: `ReportMenuTest (klass och fält)`, `ReportMenuTest.shouldPrintEachReportSectionTitleFollowedByItsRows()`, `ReportMenuTest.shouldPrintNothingToShowUnderSectionsWithoutRows()`
>
> "Överkurs för denna inlämningsuppgift"

> **7. Behållet: varför behöll ni resten av förslaget: beräkningarna i ReportService, ReportMenu med interfacet, getAllLoans, menyval 6/7 och de tio övriga testerna?**
>
> Gäller: `ReportService`, `ReportMenu.showReport()`, `ReportMenuSection`, `LoanService.getAllLoans()`, `MainMenu`, `Main.main()`, `ReportServiceTest`, `LoanServiceTest`, `EquipmentMenuDeleteTest`
>
> "Såg bra ut i sin helhet vad vi kunde se och därför vart allt kvar"

Bilaga A. Förslagets kod

Nya filer i sin helhet. Ändrade filer som diff mot grenpunkten `51fdb6e`, där rader med `+` läggs till och rader med `−` tas bort. Förslagsfilerna finns också i `~/.cache/claude-fritidslagret-rapporter/suggestion/`.

Nya filer

src/main/java/se/chasacademy/fritidslagret/service/ReportService.java

```
package se.chasacademy.fritidslagret.service;

import se.chasacademy.fritidslagret.model.equipment.Equipment;
import se.chasacademy.fritidslagret.model.loan.Loan;
import se.chasacademy.fritidslagret.model.member.Member;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class ReportService {
    private final LoanService loanService;
    private final EquipmentService equipmentService;

    public ReportService(LoanService loanService, EquipmentService equipmentService) {
        this.loanService = loanService;
        this.equipmentService = equipmentService;
    }

    public List<Loan> getActiveLoans() {
        return List.copyOf(loanService.getAllActiveLoansSortedById().values());
    }

    public List<Equipment> getMostBorrowedEquipment() {
        List<Equipment> mostBorrowedEquipment = new ArrayList<>();
        // Starts at 1 so equipment that has never been loaned can't be the most borrowed
        int highestLoanCount = 1;

        for (Equipment equipment : equipmentService.getAllEquipmentsSortedById().values()) {
            int loanCount = getLoanCountForEquipment(equipment);
            if (loanCount > highestLoanCount) {
                highestLoanCount = loanCount;
                mostBorrowedEquipment.clear();
            }
            if (loanCount == highestLoanCount) {
                mostBorrowedEquipment.add(equipment);
            }
        }
        return mostBorrowedEquipment;
    }

    public List<Equipment> getNeverBorrowedEquipment() {
        List<Equipment> neverBorrowedEquipment = new ArrayList<>();
        for (Equipment equipment : equipmentService.getAllEquipmentsSortedById().values()) {
            if (getLoanCountForEquipment(equipment) == 0) {
                neverBorrowedEquipment.add(equipment);
            }
        }
        return neverBorrowedEquipment;
    }

    public List<Member> getMembersWithMostLoans() {
        List<Member> membersWithLoans = new ArrayList<>();
        for (Loan loan : loanService.getAllLoans()) {
            if (!membersWithLoans.contains(loan.getMember())) {
                membersWithLoans.add(loan.getMember());
            }
        }
        membersWithLoans.sort(Comparator.comparing(this::getLoanCountForMember).reversed()
```

```
                .thenComparing(Member::getMemberId));
        return membersWithLoans;
    }

    public List<Equipment> getAvailableEquipment() {
        List<Equipment> availableEquipment = new ArrayList<>();
        for (Equipment equipment : equipmentService.getAllEquipmentsSortedById().values()) {
            if (equipment.isAvailable()) {
                availableEquipment.add(equipment);
            }
        }
        return availableEquipment;
    }

    public int getLoanCountForEquipment(Equipment equipment) {
        int loanCount = 0;
        for (Loan loan : loanService.getAllLoans()) {
            if (loan.getEquipment().getEquipmentId() == equipment.getEquipmentId()) {
                loanCount++;
            }
        }
        return loanCount;
    }

    public int getLoanCountForMember(Member member) {
        int loanCount = 0;
        for (Loan loan : loanService.getAllLoans()) {
            if (loan.getMember().getMemberId() == member.getMemberId()) {
                loanCount++;
            }
        }
        return loanCount;
    }
}
```

src/main/java/se/chasacademy/fritidslagret/ui/ReportMenu.java

```
package se.chasacademy.fritidslagret.ui;

import se.chasacademy.fritidslagret.service.ReportService;

import java.util.List;

public class ReportMenu {
    private final List<ReportMenuSection> reportSections;

    public ReportMenu(ReportService reportService) {
        // A new criterion becomes a new class in this list, showReport() stays the same
        this.reportSections = List.of(
                new ReportMenuActiveLoans(reportService),
                new ReportMenuMostBorrowedEquipment(reportService),
                new ReportMenuNeverBorrowedEquipment(reportService),
                new ReportMenuMembersWithMostLoans(reportService),
                new ReportMenuAvailableEquipment(reportService)
        );
    }

    void showReport() {
        IO.println("Rapport\n-----------------");
        for (ReportMenuSection section : reportSections) {
            IO.println(section.getTitle() + ":");
            List<String> rows = section.getRows();
            if (rows.isEmpty()) {
                IO.println("\tInget att visa");
            }
            for (String row : rows) {
                IO.println("\t" + row);
            }
            IO.println("");
        }
```

```
    }
}
```

src/main/java/se/chasacademy/fritidslagret/ui/ReportMenuActiveLoans.java

```
package se.chasacademy.fritidslagret.ui;

import se.chasacademy.fritidslagret.model.loan.Loan;
import se.chasacademy.fritidslagret.service.ReportService;

import java.util.ArrayList;
import java.util.List;

public class ReportMenuActiveLoans implements ReportMenuSection {
    private final ReportService reportService;

    public ReportMenuActiveLoans(ReportService reportService) {
        this.reportService = reportService;
    }

    @Override
    public String getTitle() {
        return "Aktiva lån";
    }

    @Override
    public List<String> getRows() {
        List<String> rows = new ArrayList<>();
        for (Loan loan : reportService.getActiveLoans()) {
            rows.add(loan.toString());
        }
        return rows;
    }
}
```

src/main/java/se/chasacademy/fritidslagret/ui/ReportMenuAvailableEquipment.java

```
package se.chasacademy.fritidslagret.ui;

import se.chasacademy.fritidslagret.model.equipment.Equipment;
import se.chasacademy.fritidslagret.service.ReportService;

import java.util.ArrayList;
import java.util.List;

public class ReportMenuAvailableEquipment implements ReportMenuSection {
    private final ReportService reportService;

    public ReportMenuAvailableEquipment(ReportService reportService) {
        this.reportService = reportService;
    }

    @Override
    public String getTitle() {
        return "Tillgänglig utrustning";
    }

    @Override
    public List<String> getRows() {
        List<String> rows = new ArrayList<>();
        for (Equipment equipment : reportService.getAvailableEquipment()) {
            rows.add(String.format("Id: %d - %s (%s), max %d dagar",
                    equipment.getEquipmentId(), equipment.getEquipmentName(), equipment.getType(),
                    equipment.getMaxLoanDays()));
        }
        return rows;
    }
}
```

src/main/java/se/chasacademy/fritidslagret/ui/ReportMenuMembersWithMostLoans.java

```
package se.chasacademy.fritidslagret.ui;

import se.chasacademy.fritidslagret.model.member.Member;
import se.chasacademy.fritidslagret.service.ReportService;

import java.util.ArrayList;
import java.util.List;

public class ReportMenuMembersWithMostLoans implements ReportMenuSection {
    private final ReportService reportService;

    public ReportMenuMembersWithMostLoans(ReportService reportService) {
        this.reportService = reportService;
    }

    @Override
    public String getTitle() {
        return "Medlemmar med flest lån";
    }

    @Override
    public List<String> getRows() {
        List<String> rows = new ArrayList<>();
        for (Member member : reportService.getMembersWithMostLoans()) {
            rows.add(String.format("Medlems-Id: %d - %s, %d lån",
                    member.getMemberId(), member.getName(), reportService.getLoanCountForMember(member)));
        }
        return rows;
    }
}
```

src/main/java/se/chasacademy/fritidslagret/ui/ReportMenuMostBorrowedEquipment.java

```
package se.chasacademy.fritidslagret.ui;

import se.chasacademy.fritidslagret.model.equipment.Equipment;
import se.chasacademy.fritidslagret.service.ReportService;

import java.util.ArrayList;
import java.util.List;

public class ReportMenuMostBorrowedEquipment implements ReportMenuSection {
    private final ReportService reportService;

    public ReportMenuMostBorrowedEquipment(ReportService reportService) {
        this.reportService = reportService;
    }

    @Override
    public String getTitle() {
        return "Mest utlånad utrustning";
    }

    @Override
    public List<String> getRows() {
        List<String> rows = new ArrayList<>();
        for (Equipment equipment : reportService.getMostBorrowedEquipment()) {
            rows.add(String.format("Id: %d - %s (%s), %d lån",
                    equipment.getEquipmentId(), equipment.getEquipmentName(), equipment.getType(),
                    reportService.getLoanCountForEquipment(equipment)));
        }
        return rows;
    }
}
```

src/main/java/se/chasacademy/fritidslagret/ui/ReportMenuNeverBorrowedEquipment.java

```
package se.chasacademy.fritidslagret.ui;
```

```
import se.chasacademy.fritidslagret.model.equipment.Equipment;
import se.chasacademy.fritidslagret.service.ReportService;

import java.util.ArrayList;
import java.util.List;

public class ReportMenuNeverBorrowedEquipment implements ReportMenuSection {
    private final ReportService reportService;

    public ReportMenuNeverBorrowedEquipment(ReportService reportService) {
        this.reportService = reportService;
    }

    @Override
    public String getTitle() {
        return "Utrustning som aldrig lånats ut";
    }

    @Override
    public List<String> getRows() {
        List<String> rows = new ArrayList<>();
        for (Equipment equipment : reportService.getNeverBorrowedEquipment()) {
            rows.add(String.format("Id: %d - %s (%s)",
                    equipment.getEquipmentId(), equipment.getEquipmentName(), equipment.getType()));
        }
        return rows;
    }
}
```

src/main/java/se/chasacademy/fritidslagret/ui/ReportMenuSection.java

```
package se.chasacademy.fritidslagret.ui;

import java.util.List;

public interface ReportMenuSection {
    String getTitle();

    List<String> getRows();
}
```

src/test/java/se/chasacademy/fritidslagret/service/ReportServiceTest.java

```
package se.chasacademy.fritidslagret.service;

import org.junit.jupiter.api.Test;
import se.chasacademy.fritidslagret.model.equipment.Bike;
import se.chasacademy.fritidslagret.model.equipment.Equipment;
import se.chasacademy.fritidslagret.model.equipment.SkiSet;
import se.chasacademy.fritidslagret.model.equipment.Tent;
import se.chasacademy.fritidslagret.model.loan.Loan;
import se.chasacademy.fritidslagret.model.member.Member;
import se.chasacademy.fritidslagret.model.member.MemberStatus;

import java.time.LocalDate;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

public class ReportServiceTest {

    @Test
    public void shouldCountReturnedLoansWhenCountingLoansForEquipment() {
        // Arrange
        LoanService loanService = mock(LoanService.class);
        EquipmentService equipmentService = mock(EquipmentService.class);
        ReportService reportService = new ReportService(loanService, equipmentService);
```

```
        Member member = new Member("fakeMember", MemberStatus.ACTIVE);
        Equipment bike = new Bike("fakeBike", false, 3);
        Loan returnedLoan = new Loan(member, bike, LocalDate.now(), LocalDate.now().plusDays(3), false);
        Loan activeLoan = new Loan(member, bike, LocalDate.now(), LocalDate.now().plusDays(3), true);

        when(loanService.getAllLoans()).thenReturn(List.of(returnedLoan, activeLoan));

        // Act
        int loanCount = reportService.getLoanCountForEquipment(bike);

        // Assert
        assertEquals(2, loanCount);
    }

    @Test
    public void shouldReturnEquipmentWithMostLoans() {
        // Arrange
        LoanService loanService = mock(LoanService.class);
        EquipmentService equipmentService = mock(EquipmentService.class);
        ReportService reportService = new ReportService(loanService, equipmentService);

        Member member = new Member("fakeMember", MemberStatus.ACTIVE);
        Equipment tent = new Tent("fakeTent", true, 5);
        Equipment bike = new Bike("fakeBike", true, 3);
        Loan tentLoan = new Loan(member, tent, LocalDate.now(), LocalDate.now().plusDays(5), false);
        Loan firstBikeLoan = new Loan(member, bike, LocalDate.now(), LocalDate.now().plusDays(3), false);
        Loan secondBikeLoan = new Loan(member, bike, LocalDate.now(), LocalDate.now().plusDays(3), false);

        when(equipmentService.getAllEquipmentsSortedById()).thenReturn(new TreeMap<>(Map.of(
                tent.getEquipmentId(), tent,
                bike.getEquipmentId(), bike)));
        when(loanService.getAllLoans()).thenReturn(List.of(tentLoan, firstBikeLoan, secondBikeLoan));

        // Act
        List<Equipment> mostBorrowedEquipment = reportService.getMostBorrowedEquipment();

        // Assert
        assertEquals(List.of(bike), mostBorrowedEquipment);
    }

    @Test
    public void shouldReturnAllEquipmentSharingTheHighestLoanCount() {
        // Arrange
        LoanService loanService = mock(LoanService.class);
        EquipmentService equipmentService = mock(EquipmentService.class);
        ReportService reportService = new ReportService(loanService, equipmentService);

        Member member = new Member("fakeMember", MemberStatus.ACTIVE);
        Equipment tent = new Tent("fakeTent", true, 5);
        Equipment bike = new Bike("fakeBike", true, 3);
        Equipment skis = new SkiSet("fakeSkis", true, 7);
        Loan tentLoan = new Loan(member, tent, LocalDate.now(), LocalDate.now().plusDays(5), false);
        Loan bikeLoan = new Loan(member, bike, LocalDate.now(), LocalDate.now().plusDays(3), false);

        when(equipmentService.getAllEquipmentsSortedById()).thenReturn(new TreeMap<>(Map.of(
                tent.getEquipmentId(), tent,
                bike.getEquipmentId(), bike,
                skis.getEquipmentId(), skis)));
        when(loanService.getAllLoans()).thenReturn(List.of(tentLoan, bikeLoan));

        // Act
        List<Equipment> mostBorrowedEquipment = reportService.getMostBorrowedEquipment();

        // Assert
        assertEquals(List.of(tent, bike), mostBorrowedEquipment);
    }

    @Test
    public void shouldNotReturnAnyMostBorrowedEquipmentWhenNothingHasBeenLoaned() {
        // Arrange
        LoanService loanService = mock(LoanService.class);
        EquipmentService equipmentService = mock(EquipmentService.class);
        ReportService reportService = new ReportService(loanService, equipmentService);
```

```
        Equipment tent = new Tent("fakeTent", true, 5);
        Equipment bike = new Bike("fakeBike", true, 3);

        when(equipmentService.getAllEquipmentsSortedById()).thenReturn(new TreeMap<>(Map.of(
                tent.getEquipmentId(), tent,
                bike.getEquipmentId(), bike)));
        when(loanService.getAllLoans()).thenReturn(List.of());

        // Act
        List<Equipment> mostBorrowedEquipment = reportService.getMostBorrowedEquipment();

        // Assert
        assertTrue(mostBorrowedEquipment.isEmpty());
    }

    @Test
    public void shouldReturnNeverBorrowedEquipmentEvenWhenItIsUnavailable() {
        // Arrange
        LoanService loanService = mock(LoanService.class);
        EquipmentService equipmentService = mock(EquipmentService.class);
        ReportService reportService = new ReportService(loanService, equipmentService);

        Member member = new Member("fakeMember", MemberStatus.ACTIVE);
        Equipment unavailableTent = new Tent("fakeTent", false, 5);
        Equipment returnedBike = new Bike("fakeBike", true, 3);
        Equipment skis = new SkiSet("fakeSkis", true, 7);
        Loan returnedBikeLoan = new Loan(member, returnedBike, LocalDate.now(), LocalDate.now().plusDays(3),
false);

        when(equipmentService.getAllEquipmentsSortedById()).thenReturn(new TreeMap<>(Map.of(
                unavailableTent.getEquipmentId(), unavailableTent,
                returnedBike.getEquipmentId(), returnedBike,
                skis.getEquipmentId(), skis)));
        when(loanService.getAllLoans()).thenReturn(List.of(returnedBikeLoan));

        // Act
        List<Equipment> neverBorrowedEquipment = reportService.getNeverBorrowedEquipment();

        // Assert
        assertEquals(List.of(unavailableTent, skis), neverBorrowedEquipment);
    }

    @Test
    public void shouldSortMembersWithMostLoansFirstCountingReturnedLoans() {
        // Arrange
        LoanService loanService = mock(LoanService.class);
        EquipmentService equipmentService = mock(EquipmentService.class);
        ReportService reportService = new ReportService(loanService, equipmentService);

        Member memberWithTwoActiveLoans = new Member("fakeMemberOne", MemberStatus.ACTIVE);
        Member memberWithThreeLoans = new Member("fakeMemberTwo", MemberStatus.ACTIVE);
        Equipment bike = new Bike("fakeBike", false, 3);
        Loan firstActiveLoan = new Loan(memberWithTwoActiveLoans, bike, LocalDate.now(),
LocalDate.now().plusDays(3), true);
        Loan secondActiveLoan = new Loan(memberWithTwoActiveLoans, bike, LocalDate.now(),
LocalDate.now().plusDays(3), true);
        Loan firstReturnedLoan = new Loan(memberWithThreeLoans, bike, LocalDate.now(),
LocalDate.now().plusDays(3), false);
        Loan secondReturnedLoan = new Loan(memberWithThreeLoans, bike, LocalDate.now(),
LocalDate.now().plusDays(3), false);
        Loan thirdActiveLoan = new Loan(memberWithThreeLoans, bike, LocalDate.now(),
LocalDate.now().plusDays(3), true);

        when(loanService.getAllLoans()).thenReturn(List.of(
                firstActiveLoan, secondActiveLoan, firstReturnedLoan, secondReturnedLoan, thirdActiveLoan));

        // Act
        List<Member> membersWithMostLoans = reportService.getMembersWithMostLoans();

        // Assert
        assertEquals(List.of(memberWithThreeLoans, memberWithTwoActiveLoans), membersWithMostLoans);
    }
```

```
    @Test
    public void shouldSortMembersWithSameLoanCountByMemberId() {
        // Arrange
        LoanService loanService = mock(LoanService.class);
        EquipmentService equipmentService = mock(EquipmentService.class);
        ReportService reportService = new ReportService(loanService, equipmentService);

        Member memberWithLowerId = new Member("fakeMemberOne", MemberStatus.ACTIVE);
        Member memberWithHigherId = new Member("fakeMemberTwo", MemberStatus.ACTIVE);
        Equipment bike = new Bike("fakeBike", false, 3);
        Loan loanForHigherId = new Loan(memberWithHigherId, bike, LocalDate.now(),
LocalDate.now().plusDays(3), false);
        Loan loanForLowerId = new Loan(memberWithLowerId, bike, LocalDate.now(), LocalDate.now().plusDays(3),
false);

        when(loanService.getAllLoans()).thenReturn(List.of(loanForHigherId, loanForLowerId));

        // Act
        List<Member> membersWithMostLoans = reportService.getMembersWithMostLoans();

        // Assert
        assertEquals(List.of(memberWithLowerId, memberWithHigherId), membersWithMostLoans);
    }

    @Test
    public void shouldOnlyReturnAvailableEquipment() {
        // Arrange
        LoanService loanService = mock(LoanService.class);
        EquipmentService equipmentService = mock(EquipmentService.class);
        ReportService reportService = new ReportService(loanService, equipmentService);

        Equipment availableBike = new Bike("fakeBike", true, 3);
        Equipment loanedTent = new Tent("fakeTent", false, 5);
        Equipment availableSkis = new SkiSet("fakeSkis", true, 7);

        when(equipmentService.getAllEquipmentsSortedById()).thenReturn(new TreeMap<>(Map.of(
                availableBike.getEquipmentId(), availableBike,
                loanedTent.getEquipmentId(), loanedTent,
                availableSkis.getEquipmentId(), availableSkis)));

        // Act
        List<Equipment> availableEquipment = reportService.getAvailableEquipment();

        // Assert
        assertEquals(List.of(availableBike, availableSkis), availableEquipment);
    }
}
```

src/test/java/se/chasacademy/fritidslagret/ui/ReportMenuTest.java

```
package se.chasacademy.fritidslagret.ui;

import org.junit.jupiter.api.Test;
import se.chasacademy.fritidslagret.model.equipment.Bike;
import se.chasacademy.fritidslagret.model.equipment.Equipment;
import se.chasacademy.fritidslagret.model.equipment.SkiSet;
import se.chasacademy.fritidslagret.model.equipment.Tent;
import se.chasacademy.fritidslagret.model.loan.Loan;
import se.chasacademy.fritidslagret.model.member.Member;
import se.chasacademy.fritidslagret.model.member.MemberStatus;
import se.chasacademy.fritidslagret.service.EquipmentService;
import se.chasacademy.fritidslagret.service.LoanService;
import se.chasacademy.fritidslagret.service.ReportService;

import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.time.LocalDate;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
```

```java
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

public class ReportMenuTest {
    /*
    Not named *UI* on purpose: the test never walks through Main.main or reads System.in.
    It creates Member and Equipment objects, which moves the static id counters. If it ran
    before EquipmentMenuDeleteTest in the ui-test runtime, Startup would not find member 1000
    and Main.main would crash with a NullPointerException.
    */

    @Test
    public void shouldPrintEachReportSectionTitleFollowedByItsRows() {
        // Save Default Output Stream !!!! IMPORTANT !!!!
        PrintStream stdOut = System.out;

        try {
            // Arrange
            LoanService loanService = mock(LoanService.class);
            EquipmentService equipmentService = mock(EquipmentService.class);
            ReportService reportService = new ReportService(loanService, equipmentService);
            ReportMenu reportMenu = new ReportMenu(reportService);

            Member member = new Member("fakeMember", MemberStatus.ACTIVE);
            Equipment tent = new Tent("fakeTent", false, 5);
            Equipment bike = new Bike("fakeBike", false, 3);
            Equipment skis = new SkiSet("fakeSkis", true, 7);
            Loan returnedBikeLoan = new Loan(member, bike, LocalDate.now(), LocalDate.now().plusDays(3),
false);
            Loan activeBikeLoan = new Loan(member, bike, LocalDate.now(), LocalDate.now().plusDays(3), true);
            Loan returnedSkisLoan = new Loan(member, skis, LocalDate.now(), LocalDate.now().plusDays(7),
false);

            when(loanService.getAllLoans()).thenReturn(List.of(returnedBikeLoan, activeBikeLoan,
returnedSkisLoan));
            when(loanService.getAllActiveLoansSortedById()).thenReturn(new TreeMap<>(Map.of(
                    activeBikeLoan.getLoanId(), activeBikeLoan)));
            when(equipmentService.getAllEquipmentsSortedById()).thenReturn(new TreeMap<>(Map.of(
                    tent.getEquipmentId(), tent,
                    bike.getEquipmentId(), bike,
                    skis.getEquipmentId(), skis)));

            String newLine = System.lineSeparator();
            ByteArrayOutputStream consoleOutput = new ByteArrayOutputStream();
            System.setOut(new PrintStream(consoleOutput));

            // Act
            reportMenu.showReport();

            // Assert
            String report = consoleOutput.toString();
            assertTrue(report.contains("Aktiva lån:" + newLine
                    + "\t" + activeBikeLoan + newLine + newLine));
            assertTrue(report.contains("Mest utlånad utrustning:" + newLine
                    + "\tId: " + bike.getEquipmentId() + " - fakeBike (Bike), 2 lån" + newLine + newLine));
            assertTrue(report.contains("Utrustning som aldrig lånats ut:" + newLine
                    + "\tId: " + tent.getEquipmentId() + " - fakeTent (Tent)" + newLine + newLine));
            assertTrue(report.contains("Medlemmar med flest lån:" + newLine
                    + "\tMedlems-Id: " + member.getMemberId() + " - fakeMember, 3 lån" + newLine + newLine));
            assertTrue(report.contains("Tillgänglig utrustning:" + newLine
                    + "\tId: " + skis.getEquipmentId() + " - fakeSkis (SkiSet), max 7 dagar" + newLine +
newLine));

        } finally {
            // Restore Default Output Stream !!!! IMPORTANT !!!!
            System.setOut(stdOut);
        }
    }

    @Test
    public void shouldPrintNothingToShowUnderSectionsWithoutRows() {
        // Save Default Output Stream !!!! IMPORTANT !!!!
        PrintStream stdOut = System.out;
```

```
        try {
            // Arrange
            LoanService loanService = mock(LoanService.class);
            EquipmentService equipmentService = mock(EquipmentService.class);
            ReportService reportService = new ReportService(loanService, equipmentService);
            ReportMenu reportMenu = new ReportMenu(reportService);

            when(loanService.getAllLoans()).thenReturn(List.of());
            when(loanService.getAllActiveLoansSortedById()).thenReturn(new TreeMap<>());
            when(equipmentService.getAllEquipmentsSortedById()).thenReturn(new TreeMap<>());

            String newLine = System.lineSeparator();
            ByteArrayOutputStream consoleOutput = new ByteArrayOutputStream();
            System.setOut(new PrintStream(consoleOutput));

            // Act
            reportMenu.showReport();

            // Assert
            String report = consoleOutput.toString();
            assertTrue(report.contains("Aktiva lån:" + newLine + "\tInget att visa" + newLine));
            assertTrue(report.contains("Mest utlånad utrustning:" + newLine + "\tInget att visa" + newLine));
            assertTrue(report.contains("Utrustning som aldrig lånats ut:" + newLine + "\tInget att visa" +
newLine));
            assertTrue(report.contains("Medlemmar med flest lån:" + newLine + "\tInget att visa" + newLine));
            assertTrue(report.contains("Tillgänglig utrustning:" + newLine + "\tInget att visa" + newLine));

        } finally {
            // Restore Default Output Stream !!!! IMPORTANT !!!!
            System.setOut(stdOut);
        }
    }
}
```

Ändrade filer

src/main/java/se/chasacademy/fritidslagret/Main.java

```
--- före
+++ förslaget
@@ -26,12 +26,13 @@

         LoanService loanService = new LoanService(loanRepository, memberService, equipmentService,
waitingListService, clock);
         StatisticsService statisticsService = new StatisticsService(equipmentService, loanService,
memberService);
+        ReportService reportService = new ReportService(loanService, equipmentService);

         // Startup
         Startup startup = new Startup(equipmentService, memberService, loanService);
         startup.startUp();
         //fritidslagret.se.chasacademy.fritidslagret.se.chasacademy.fritidslagret.Main-Menu-loop
-        MainMenu mainMenu = new MainMenu(loanService, memberService, equipmentService, statisticsService,
waitingListService);
+        MainMenu mainMenu = new MainMenu(loanService, memberService, equipmentService, statisticsService,
waitingListService, reportService);
         mainMenu.mainMenu();
     }
 }
```

src/main/java/se/chasacademy/fritidslagret/service/LoanService.java

```
--- före
+++ förslaget
@@ -120,6 +120,10 @@
                 .toList();
     }

+    public List<Loan> getAllLoans() {
```

```
+        return List.copyOf(loanRepository.getLoanDatabase().values());
+    }
+
     public SortedMap<Integer, Loan> getAllActiveLoansSortedById() {
         SortedMap<Integer, Loan> allActiveLoans = new TreeMap<>();
         for (Loan loan : loanRepository.getLoanDatabase().values()) {
```

src/main/java/se/chasacademy/fritidslagret/ui/MainMenu.java

```
--- före
+++ förslaget
@@ -3,6 +3,7 @@
 import se.chasacademy.fritidslagret.service.EquipmentService;
 import se.chasacademy.fritidslagret.service.LoanService;
 import se.chasacademy.fritidslagret.service.MemberService;
+import se.chasacademy.fritidslagret.service.ReportService;
 import se.chasacademy.fritidslagret.service.WaitingListService;
 import se.chasacademy.fritidslagret.service.StatisticsService;

@@ -12,14 +13,16 @@
     private final MemberMenu memberMenu;
     private final StatisticsMenu statisticsMenu;
     private final WaitingListMenu waitingListMenu;
+    private final ReportMenu reportMenu;


-    public MainMenu(LoanService loanService, MemberService memberService, EquipmentService equipmentService,
StatisticsService statisticsService, WaitingListService waitingListService) {
+    public MainMenu(LoanService loanService, MemberService memberService, EquipmentService equipmentService,
StatisticsService statisticsService, WaitingListService waitingListService, ReportService reportService) {
         this.equipmentMenu = new EquipmentMenu(equipmentService);
         this.memberMenu = new MemberMenu(memberService);
         this.loanMenu = new LoanMenu(loanService, equipmentService, memberService);
         this.statisticsMenu = new StatisticsMenu(statisticsService);
         this.waitingListMenu = new WaitingListMenu(waitingListService);
+        this.reportMenu = new ReportMenu(reportService);

     }

@@ -34,7 +37,8 @@
                         3. Lån
                         4. Statistik
                         5. Väntelistor
-                        6. Avsluta
+                        6. Rapporter
+                        7. Avsluta
                         """);

                 // Input main-menu-choice
@@ -47,7 +51,8 @@
                     case 3 -> loanMenu.loanMenu();
                     case 4 -> statisticsMenu.statisticsMenu();
                     case 5 -> waitingListMenu.waitingListMenu();
-                    case 6 -> {
+                    case 6 -> reportMenu.showReport();
+                    case 7 -> {
                         return;
                     }
                 }
```

src/test/java/se/chasacademy/fritidslagret/service/LoanServiceTest.java

```
--- före
+++ förslaget
@@ -18,6 +18,7 @@
 import java.util.HashMap;
 import java.util.List;
 import java.util.Map;
+import java.util.SortedMap;

 import static org.junit.jupiter.api.Assertions.*;
```

```
 import static org.mockito.Mockito.*;
@@ -411,4 +412,61 @@
         // Assert
         assertEquals(5, delayedDays);
     }
+
+    @Test
+    public void shouldIncludeReturnedLoansWhenGettingAllLoans() {
+        // Arrange
+        LoanRepository loanRepository = mock(LoanRepository.class);
+        MemberService memberService = mock(MemberService.class);
+        EquipmentService equipmentService = mock(EquipmentService.class);
+        WaitingListService waitingListService = mock(WaitingListService.class);
+        Clock clock = Clock.systemUTC();
+
+        LoanService loanService = new LoanService(loanRepository, memberService, equipmentService,
waitingListService, clock);
+
+        Member member = new Member("fakeName", MemberStatus.ACTIVE);
+        Equipment equipment = new SkiSet("fakeName", true, 3);
+        Loan activeLoan = new Loan(member, equipment, LocalDate.now(), LocalDate.now().plusDays(3), true);
+        Loan returnedLoan = new Loan(member, equipment, LocalDate.now(), LocalDate.now().plusDays(3),
false);
+
+        when(loanRepository.getLoanDatabase()).thenReturn(Map.of(
+                activeLoan.getLoanId(), activeLoan,
+                returnedLoan.getLoanId(), returnedLoan));
+
+        // Act
+        List<Loan> allLoans = loanService.getAllLoans();
+
+        // Assert
+        assertEquals(2, allLoans.size());
+        assertTrue(allLoans.contains(returnedLoan));
+    }
+
+    @Test
+    public void shouldNotListReturnedLoansAsActiveLoans() {
+        // Arrange
+        LoanRepository loanRepository = mock(LoanRepository.class);
+        MemberService memberService = mock(MemberService.class);
+        EquipmentService equipmentService = mock(EquipmentService.class);
+        WaitingListService waitingListService = mock(WaitingListService.class);
+        Clock clock = Clock.systemUTC();
+
+        LoanService loanService = new LoanService(loanRepository, memberService, equipmentService,
waitingListService, clock);
+
+        Member member = new Member("fakeName", MemberStatus.ACTIVE);
+        Equipment equipment = new SkiSet("fakeName", true, 3);
+        Loan firstActiveLoan = new Loan(member, equipment, LocalDate.now(), LocalDate.now().plusDays(3),
true);
+        Loan returnedLoan = new Loan(member, equipment, LocalDate.now(), LocalDate.now().plusDays(3),
false);
+        Loan secondActiveLoan = new Loan(member, equipment, LocalDate.now(), LocalDate.now().plusDays(3),
true);
+
+        when(loanRepository.getLoanDatabase()).thenReturn(Map.of(
+                firstActiveLoan.getLoanId(), firstActiveLoan,
+                returnedLoan.getLoanId(), returnedLoan,
+                secondActiveLoan.getLoanId(), secondActiveLoan));
+
+        // Act
+        SortedMap<Integer, Loan> activeLoans = loanService.getAllActiveLoansSortedById();
+
+        // Assert
+        assertEquals(List.of(firstActiveLoan, secondActiveLoan), List.copyOf(activeLoans.values()));
+    }
 }
```

src/test/java/se/chasacademy/fritidslagret/ui/EquipmentMenuDeleteTest.java

```
--- före
+++ förslaget
@@ -41,7 +41,7 @@
                     "1\n" +          // Choose Equipment
                     "abcdefgh\n" +    // Now standing at confirmationDialog
                     "5\n" +          // Going out of menu
-                    "6\n"            // Exit Program
+                    "7\n"            // Exit Program
                     ;

             ByteArrayOutputStream consoleOutput = new ByteArrayOutputStream();
```

Bilaga B. Hur watchers fungerar

Watcher 1	`watch/watcher.py` körs i bakgrunden (Claude Code `Monitor`). Var tredje sekund jämförs filernas ändringstider. Efter 45 sekunder utan nya sparningar körs jämförelsen (`watch/compare.py`), och en rad skrivs när någon enhet byter status. Commits, pushar och grenbyten ger också en rad. Testerna körs i en ögonblicksbild (`git archive`) av varje commit, aldrig i projektmappen.
Watcher 2	En bakgrundsloop som väntar på `watch/phase1-candidate` (skrivs av watcher 1) eller `watch/phase1-done` (när ni säger att ni är klara). När filen dyker upp ställer Claude frågorna om varför. Svaren sparas i `watch/answers.json` och PDF:en byggs om.
Vad som läses	Arbetskopian i `~/Skola/ChasProjects/se.chasacademy.fritidslagret` och git, utan lås (`git --no-optional-locks`: `rev-parse`, `show`, `ls-tree`, `status`, `archive`). Inget i projektet eller i `.git` skrivs.
Status just nu	`~/.cache/claude-fritidslagret-rapporter/watch/status.md`, som uppdateras vid varje jämförelse. Historiken finns i `history.jsonl`.